



Azure Functions

✓ 1. Azure Functions 개요

- 정의

Azure Functions는 **서버리스 컴퓨팅(Serverless Computing)** 기반의 클라우드 서비스로, 이벤트 기반으로 작동하며 사용자는 인프라를 신경 쓰지 않고 코드 작성에만 집중할 수 있다.

- 특징

- 필요할 때만 실행되는 함수
- 사용한 만큼만 요금 부과 (Consumption Plan)
- 다양한 Azure 서비스 및 외부 시스템과 연동 가능
- 자동 확장(Auto-Scaling) 지원

- 활용 예시

- 웹 API 서버
- IoT 데이터 처리
- 메시지 큐 기반 처리
- 예약 작업 자동화
- AI 서비스 연동

✓ 2. Azure Functions 주요 구성 요소

트리거 (Trigger)

트리거 종류	설명
HTTP 트리거	API 요청 시 함수 실행
Timer 트리거	특정 시간에 주기적으로 실행
Blob 트리거	Blob Storage에 파일 업로드 시 실행
Queue 트리거	메시지 큐에 메시지 추가 시 실행
Event Hubs 트리거	스트리밍 데이터 수신 시 실행
Cosmos DB 트리거	Cosmos DB의 변경 감지 시 실행

바인딩 (Binding)

- **입력 바인딩 (Input Binding):** 함수 실행 시 외부 리소스로부터 데이터를 자동으로 가져옴

- **출력 바인딩 (Output Binding):** 함수 실행 후 외부 리소스에 결과를 자동으로 전달

예시 (`function.json`):

```
{
  "bindings": [
    {
      "name": "inputBlob",
      "type": "blobTrigger",
      "direction": "in",
      "path": "sample-images/{name}",
      "connection": "AzureWebJobsStorage"
    },
    {
      "name": "outputBlob",
      "type": "blob",
      "direction": "out",
      "path": "thumbnails/{name}",
      "connection": "AzureWebJobsStorage"
    }
  ]
}
```

✅ 3. Azure Functions 주요 활용 분야

- 파일 업로드 처리 자동화
- 실시간 데이터 스트림 처리
- 예약 기반 작업 실행
- AI 연동 감정 분석
- 확장 가능한 REST API 개발
- 복잡한 상태 기반 워크플로우(Durable Functions)
- Cosmos DB 트리거 기반 이상 감지
- 메시지 기반 티켓 예매 시스템

✅ 4. 실전 시나리오별 설명 및 코드 예시

1) 파일 업로드 및 처리 자동화

설명

파트너 시스템이 Blob Storage에 제품 파일 업로드 → 트리거 작동 → 유효성 검사 및 변환 → 시스템 전달

전체 흐름

1. Blob Storage에 업로드
2. 트리거 감지 → 함수 실행
3. 유효성 검사 → CSV → JSON 변환
4. 처리 결과 저장 또는 API로 전달

2) AI 서비스 연동 - 감정 분석

설명

Text Analytics API를 통해 HTTP로 텍스트를 받아 감정(긍정/부정/중립) 분석 후 저장

예시 코드 (Python):

```
import requests
import azure.functions as func

def main(req: func.HttpRequest) → func.HttpResponse:
    text = req.params.get('text')
    endpoint = "https://<region>.api.cognitive.microsoft.com/text/analytics/v3.0/sentiment"
    headers = {
        "Ocp-Apim-Subscription-Key": "<API_KEY>",
        "Content-Type": "application/json"
    }
    body = {"documents": [{"id": "1", "language": "en", "text": text}]}
    response = requests.post(endpoint, headers=headers, json=body)
    return func.HttpResponse(response.text, mimetype="application/json")
```

3) IoT 실시간 데이터 처리

설명

IoT 센서 → Event Hubs 수집 → Azure Functions 전처리 → Stream Analytics로 분석 및 대시보드

4) 예약 작업 자동화 (Timer Trigger)

설명

Timer 트리거를 활용해 금융 서비스 고객 데이터에서 중복 항목을 제거

function.json 예시:

```
{
  "bindings": [
    {
      "name": "myTimer",
      "type": "timerTrigger",
      "direction": "in",
      "schedule": "0 0 2 * * *"
    }
  ]
}
```

5) 확장 가능한 API 구축

설명

POST /api/register로 사용자 정보를 받아 검증 후 저장

기본 코드 흐름:

- HTTP 요청 수신
- JSON 유효성 검사
- 비밀번호 해시 처리
- Cosmos DB 또는 SQL에 저장
- 성공 응답 반환

6) Durable Functions로 워크플로 구현

설명

복잡한 주문 처리 로직: 주문 접수 → 결제 → 재고 확인 → 배송 준비를 단계별로 함수 분리하여 순차 실행

7) Cosmos DB 트리거 + 이상 거래 감지

설명

거래 정보가 Cosmos DB에 저장될 때마다 트리거 함수 실행 → 이상 거래 탐지 → 알림 전송

8) 메시지 기반 티켓 예매 시스템

흐름

1. HTTP 트리거 → 사용자 요청 수신
2. Azure Queue에 메시지 저장
3. Queue 트리거 함수 실행

4. 유효성 검증, 결제 처리
 5. 결과 저장 및 사용자에게 이메일 발송
-

✓ 5. 개발 환경 구축 가이드

1. VS Code 설치

<https://code.visualstudio.com>

2. Python 설치 (3.11.9)

<https://www.python.org/downloads>

- 설치 시 **Add Python.exe to PATH** 체크 필수

3. .NET SDK 설치 (8.0)

<https://dotnet.microsoft.com/ko-kr/download>

4. Azure Functions Core Tools 설치

<https://github.com/azure/azure-functions-core-tools>

5. VS Code 확장 프로그램 설치

- Azure Functions
 - Azurite (로컬 Blob Storage 시뮬레이터)
 - Python
-

✓ 6. 요약

- Azure Functions는 서버리스 컴퓨팅 환경에서 이벤트 기반 처리 자동화를 가능하게 하는 플랫폼이다.
- 다양한 트리거와 바인딩을 활용해 실시간 처리, 백엔드 API, 예약작업, 파일처리 등 폭넓은 기능을 구현할 수 있다.
- 실무 예제들을 통해 실제 업무 자동화 및 시스템 연동에서 광범위하게 사용 가능하다.
- VS Code, Python, .NET SDK, Core Tools로 간편하게 로컬 개발 환경을 구축할 수 있다.